

Reviewer Pack — Minimal Order of Grothendieck-Teichmüller Groups and Circuit...

Entry c6f0f1d005fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:08:37 UTC

Minimal Order of Grothendieck-Teichmüller Groups and Circuit Depth Inequality

Entry ID: c6f0f1d005fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 16:08:37 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Grothendieck-Teichmüller Groups) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Depth)

Statement:

For every Boolean circuit C with depth D , the minimal order of the Grothendieck-Teichmüller group associated with C 's clause set is at most D^3 .

Rationale (proposer's reasoning):

The Grothendieck-Teichmüller groups capture deep arithmetic structures that may encode complexity-theoretic invariants such as circuit depth. The conjecture proposes a connection between these groups and circuit complexity, which could expose new structural insights into computational hardness.

Taxonomy category: Arithmetic Geometry × Boolean Circuit Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ded192cef40ec1f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean circuit C , if the minimal order of the Grothendieck-Teichmüller group associated with C 's clause set is less than or equal to D^3 where D is the depth of the circuit, then the conjecture is supported. Otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck-Teichmüller Groups" AND "circuit depth" - "arithmetic geometry" AND minimal order circuit complexity - "Boolean circuit complexity" AND Teichmüller groups AND depth inequality

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_circuit(n_inputs, depth):
    if depth == 1:
        return [f'x{i}' for i in range(n_inputs)]

    left = generate_random_circuit(n_inputs, depth - 1)
    right = generate_random_circuit(n_inputs, depth - 1)

    return [f'({l} & {r})' for l in left] + [f'({l} | {r})' for l in right]

def compute_clause_set(circuit):
    if isinstance(circuit, str):
        return set([circuit])
    else:
        clause_set = set()
        for part in circuit:
            clause_set.update(compute_clause_set(part))
        return clause_set

def cohomological_construction(clause_set):
    # Simplified version of the cohomological construction
    # This is a placeholder and should be replaced with actual computation
    return len(clause_set)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n_inputs in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            circuit = generate_random_circuit(n_inputs, depth=random.randint(1, 10))
```

```

        clause_set = compute_clause_set(circuit)
        grothendieck_order = cohomological_construction(clause_set)
        depth = random.randint(1, 10)

        results.append({
            "n": n_inputs,
            "depth": depth,
            "grothendieck_order": grothendieck_order
        })

    if not results:
        return {
            "metric_name": "Grothendieck Order vs Depth",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No circuits generated"
        }

    metric_values = [r["grothendieck_order"] for r in results]
    n_max = max(r["n"] for r in results)
    conjecture_holds = all(grothendieck_order <= depth**3 for grothendieck_order, depth in zip(metric_values, n_max))

    return {
        "metric_name": "Grothendieck Order vs Depth",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "First failing instance"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"First failing instance\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 47, in run_trial
    circuit = generate_random_circuit(n_inputs, depth=random.randint(1, 10))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 22, in generate_rand
    left = generate_random_circuit(n_inputs, depth - 1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 22, in generate_rand
    left = generate_random_circuit(n_inputs, depth - 1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 22, in generate_rand
    left = generate_random_circuit(n_inputs, depth - 1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 3 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_15156e88.py", line 25, in generate_rand
    return [f'({l} & {r})' for l in left] + [f'({l} | {r})' for l in right]
           ^
NameError: name 'r' is not defined. Did you mean: 're'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13698
2	preregistration	ollama_remote	glm4:latest	0	0	9035
3	novelty	ollama_remote	glm4:latest	0	0	9069
4	novelty	ollama_remote	glm4:latest	0	0	13844
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13313
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9983
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9574
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10092
9	judge	ollama_remote	glm4:latest	0	0	11517

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
---	-------	----------	-------	-----------	-----	--------------

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 100125 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c6f0f1d005fc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c6f0f1d005fc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c6f0f1d005fc.tar.gz (if generated)